

Take-Home Exercise: AI-Native Multi-Agent Workflow with Agno

Objective

Build a practical multi-agent system using **Agno Workflows or Teams** to automate a realistic business or operational workflow.

This exercise is intentionally designed for **AI-native / vibe-coding development**. You are encouraged to use AI-assisted tools as much as possible—such as **Codex, Claude Code, Cursor, Antigravity, Copilot, or similar tools**—so long as you can explain your design choices, debug issues, and defend the final system.

Your goal is not to avoid AI. Your goal is to **use AI effectively to ship a solid working system**.

Choose one track

Pick **one** of the following:

Option A — Browser Automation Multi-Agent

Build a team that completes a browser-based task on a public website and returns structured results.

Possible examples:

- search and extract top results from a public directory
- navigate a multi-step public workflow
- collect structured information across several pages

Suggested team:

- Planner Agent
 - Browser Agent
 - Extraction Agent
 - Verifier Agent
-

Option B — Marketing Team

Build a team that turns a simple product brief into a set of usable marketing assets.

Possible examples:

- landing page copy
- launch email sequence
- ad concepts
- social content calendar

Suggested team:

- Strategist Agent
 - Research Agent
 - Copywriter Agent
 - Editor/QA Agent
-

Option C — Investment Team

Build a team that researches and evaluates a small set of companies, sectors, or opportunities and produces a recommendation memo.

Possible examples:

- compare 3 AI startups in a niche
- compare 3 public companies in a sector
- create a short investment memo with risks and open questions

Suggested team:

- Research Agent
 - Analyst Agent
 - Critic/Risk Agent
 - Decision Agent
-

Option D — Operators Team (recommended: Revenue Operations)

Build a team that helps an operator prioritize work and recommend actions.

Recommended vertical: **Revenue Operations / Sales Ops**

Possible examples:

- prioritize leads or accounts
- flag at-risk pipeline opportunities
- recommend follow-up actions

- summarize account notes into an operator dashboard

Suggested team:

- Intake Agent
 - Classification/Enrichment Agent
 - Action Agent
 - Review/Manager Agent
-

Core requirements

1. Multi-agent orchestration

Your system must use a real multi-agent pattern, not just one agent with a long prompt.

Examples of acceptable patterns:

- planner → specialists → reviewer
- hierarchical coordinator + workers
- generator → critic → revision
- router → specialist agents → aggregator

You should have at least:

- **1 coordinating agent**
 - **2 specialist agents**
 - **1 output/review step**
-

2. Runnable end-to-end workflow

The system should be runnable locally and produce a real output for your chosen track.

Examples:

- structured JSON
 - markdown report
 - CSV of ranked results
 - generated campaign package
 - investment memo
 - prioritized ops action list
-

3. Typed state and clean interfaces

Use strongly typed objects for agent inputs/outputs and workflow state.

Examples:

- Pydantic models
- typed dataclasses
- explicit schemas for handoffs

Avoid “stringly typed” handoffs wherever possible.

4. Observability and instrumentation

Track at least:

- total workflow latency
- per-agent latency
- token usage, if available
- success/failure status

A simple logging table or structured JSON log is sufficient.

5. Resilience

Include at least **two failure-handling scenarios**.

Examples:

- malformed user input
 - missing or partial source data
 - transient LLM/tool failure with retry
 - browser/navigation failure
 - invalid structured output that triggers repair/retry
-

6. Demo UX

Provide a basic way to run and inspect the workflow.

Preferred:

- **Agno Agent OS UI**

Acceptable fallback:

- CLI + readable logs
- minimal web UI
- notebook with reproducible run steps

If you use Agent OS UI, show the team execution clearly.

Strong hint on implementation

Do not overbuild this.

A good submission can be:

- a small but clean Agno workflow
- 4 agents
- 2–4 tools
- typed state
- basic retries
- one good demo scenario
- one or two focused tests

Simple, working, and well-explained beats ambitious but broken.

Suggested project structure

```
repo/  
  README.md  
  app/  
    agents/  
    workflows/  
    models/  
    tools/  
    tests/  
  data/ or examples/  
  demo/
```

Deliverables

1. Repository

A GitHub repository with clear setup and run instructions.

2. README

Include:

- which track you chose
- the agent architecture
- what each agent does
- tools used
- where AI-assisted coding helped
- tradeoffs and known limitations

3. Demo

Submit one of:

- short screen recording
- Agent OS UI walkthrough
- terminal demo with logs

4. Example output

Include at least one representative output artifact:

- report
- ranked list
- structured JSON
- campaign package
- memo
- action plan

5. Brief build notes

In 5–15 bullet points, explain:

- what AI tools you used
- where they sped you up
- where they failed or needed correction
- what you personally changed/debugged

This is important. We want to see how you work with AI, not whether you avoided it.

Optional stretch goals

These are not required, but good differentiators:

- **Self-correction loop:** reviewer sends work back for one or more repair iterations
- **Human-in-the-loop breakpoint:** require an approval step before final output
- **Parallel specialists:** run multiple workers concurrently

- **Evaluation harness:** compare outputs against a small rubric or expected schema
- **Trace visualization:** show handoffs, decisions, or state transitions

These mirror the stronger elements in the original exercise's reviewer loop, observability, and manual approval concepts.

How we will grade it

We score across 10 dimensions. Each dimension gets a score from **1 to 10**. We then look at the overall shape of the submission, not just the average.

1. Working end-to-end system

- 1–3: incomplete, brittle, or does not run
- 4–6: runs partially; core workflow works with manual fixes
- 7–8: runs end-to-end reliably for the intended demo case
- 9–10: polished, repeatable, and clearly usable

2. Multi-agent design quality

- 1–3: agents are mostly fake wrappers around one prompt
- 4–6: some separation of responsibilities, but weak orchestration
- 7–8: clear delegation and meaningful handoffs
- 9–10: elegant architecture with justified coordination pattern

3. Product / workflow sense

- 1–3: output is technically generated but not very useful
- 4–6: useful in parts, but misses practical workflow needs
- 7–8: good real-world framing and useful outputs
- 9–10: strong operator/product judgment; feels genuinely valuable

4. Code quality and maintainability

- 1–3: messy, hard to follow, weak structure
- 4–6: acceptable but uneven
- 7–8: clean organization, readable abstractions
- 9–10: strong software taste; easy to extend

5. Typed state and interfaces

1–3: mostly unstructured strings and fragile parsing
4–6: partial schemas; some implicit contracts
7–8: good typed models and explicit handoffs
9–10: disciplined, consistent schema-first design

6. Reliability and error handling

1–3: failure cases largely ignored
4–6: minimal retries or validation
7–8: reasonable repair/retry paths and validation
9–10: robust and thoughtfully defensive

7. Observability and instrumentation

1–3: little or no visibility into execution
4–6: basic logs only
7–8: useful latency/status/tool traces
9–10: strong execution visibility and helpful debugging signals

8. Use of AI tools / vibe coding effectiveness

1–3: appears copy-pasted without understanding
4–6: used AI tools, but weak ownership of the result
7–8: good leverage of AI with clear human debugging and iteration
9–10: excellent AI-native workflow; candidate obviously accelerated themselves while staying in control

9. Explanation and tradeoff clarity

1–3: poor README, little reasoning
4–6: some explanation, but shallow
7–8: clear design rationale and limitations
9–10: thoughtful, concise, and technically persuasive

10. Overall engineering judgment

1–3: weak prioritization; wrong things built
4–6: decent choices, but some wasted effort
7–8: good prioritization and scope control
9–10: excellent “startup engineer” judgment

Interpreting the final score

You can summarize overall performance like this:

- **9–10**: exceptional; strong hire signal
- **7–8**: solid; likely worth next-round interview
- **5–6**: mixed; may be promising but needs probing
- **1–4**: weak signal for this role

A candidate does not need a 9+ on everything. For an AI-native startup engineer, the strongest signals are usually:

- end-to-end execution
- multi-agent decomposition
- practical judgment
- reliability
- effective use of AI tools without losing control

Appendix

- If you need AI credits, [see](#)

Update: